

cartographer Relocalization Navigation

cartographer Relocalization Navigation

1. Course Content
2. Principle Introduction
 - 2.1 Introduction
 - 2.2 Differences Between the Two Localization Methods
3. Demo
 - 3.1 Preparation
 - 3.2 Starting the Navigation Node
4. Source Code Analysis
 - 4.1 Viewing the TF Tree
 - 4.2 Launch File

1. Course Content

1. Run the demo program. The robot can quickly determine its position on the map near its initial location and then perform navigation

2. Principle Introduction

2.1 Introduction

- Navigation 2's default localization algorithm is AMCL (Adaptive Monte Carlo Localization). This replaces the default AMCL localization method with cartographer localization, while all other navigation functions remain unchanged.

2.2 Differences Between the Two Localization Methods

- **AMCL:** suitable for scenarios with a known map. For example, in an indoor environment where a map has already been built, AMCL can be used to determine the robot's position when it performs repetitive tasks or navigation.
- **Cartographer:** more suitable for unknown environments where the robot needs to map and localize simultaneously. For example, when a robot enters a new area for the first time and needs to quickly build a map while determining its own position, Cartographer can perform better.

3. Demo

3.1 Preparation

- You need an occupancy grid map and a pose graph map in pstream format. To build a pstream pose graph map, refer to the [7. Cartographer Mapping](#) chapter



map.pbstream



map.pgm



map.yaml

[!CAUTION]

Note that the occupancy grid map and the pbstream pose graph map must correspond to each other.

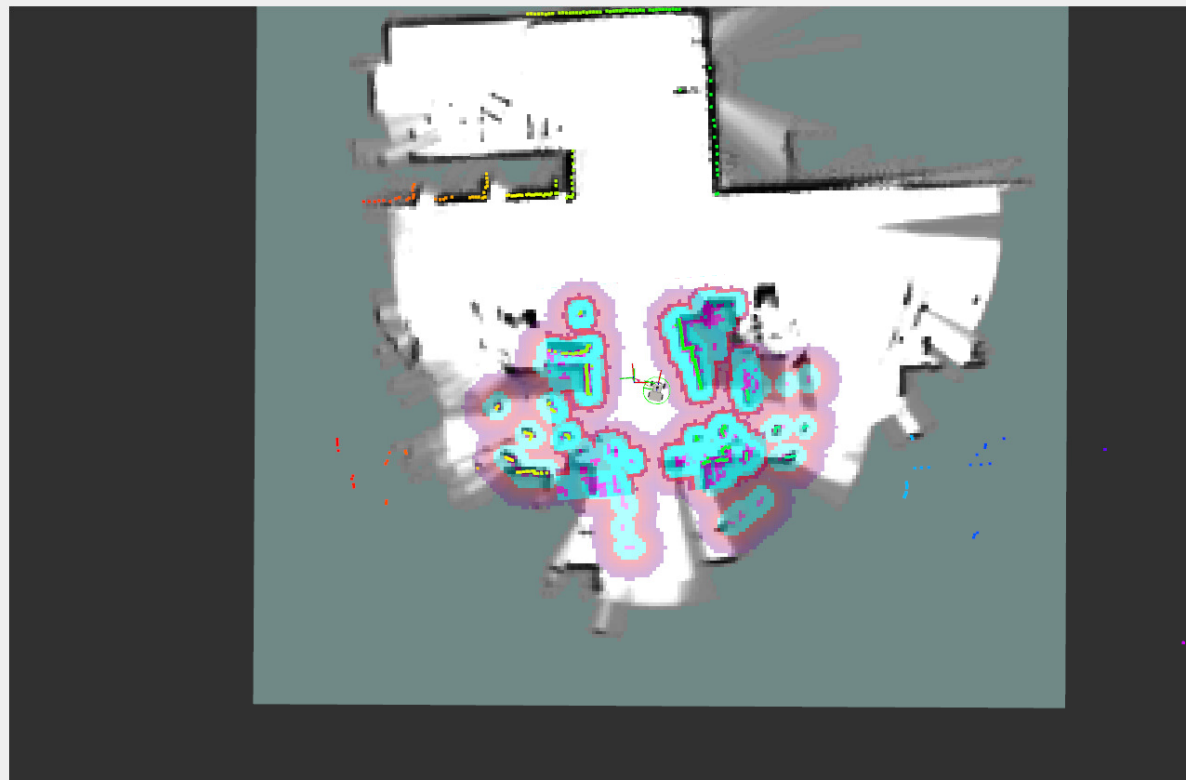
3.2 Starting the Navigation Node

```
ros2 launch microros_v2_bringup nav2.launch.py localization_mode:=cartographer
```

[!TIP]

Launch parameter descriptions

- localization_mode
 - The global localization method; here cartographer localization is selected
 - state_filename
 - The name of the pbstream pose graph map. If cartographer is selected as the localization method for navigation, the pose graph map name must be passed in; the default value is map.pbstream
-
- In rviz you can see that the robot automatically completes the initial position estimation without manual position initialization. The remaining navigation steps are the same as when using the AMCL localization method normally.



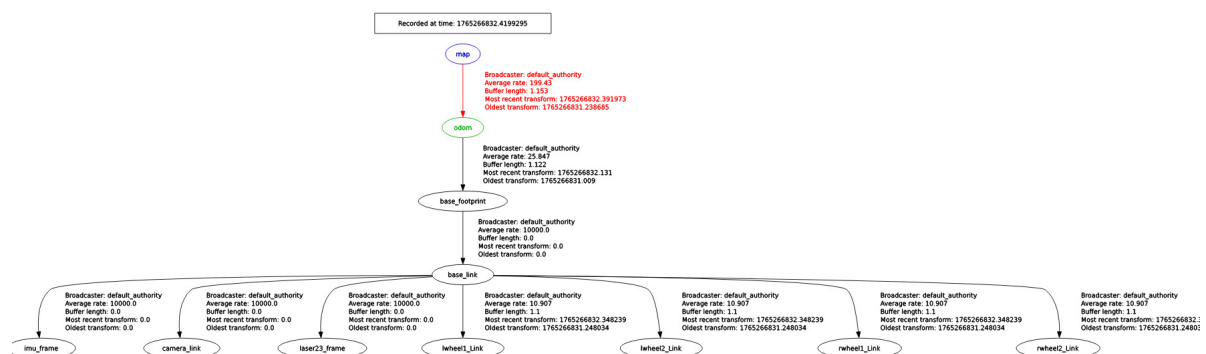
4. Source Code Analysis

4.1 Viewing the TF Tree

In the terminal, run:

```
ros2 run rqt_tf_tree rqt_tf_tree
```

- Here, the TF transform from the map frame to the odom frame in the TF tree is provided by cartographer.



4.2 Launch File

```
$HOME/microros_ws/src/microros_v2_bringup/launch/nav2.launch.py
```

- The key program snippet is as follows; it starts the navigation2 navigation stack node and the cartographer pure localization node

```
#Cartographer positioning navigation
cartographer_localization_nav = GroupAction(
    condition=IfCondition(PythonExpression(["'", localization_mode, "' ==
'cartographer' and '", slam, "' == 'False' "]])),
    actions=[
        #Start navigation2 navigation file (no global positioning)
        PushRosNamespace(namespace=namespace),
        IncludeLaunchDescription(
            PythonLaunchDescriptionSource(nav2_launch_mod),
            launch_arguments={
                "map": map_file,
                "params_file": params_file,
                "use_sim_time": "False",
                "namespace": namespace,
                "log_level": log_level,
            }.items()),
        Node(
            package = 'cartographer_ros',
            executable = 'cartographer_node',
            parameters = [{'use_sim_time': False}],
            arguments = [
                '-configuration_directory',
                os.path.join(get_package_share_directory('microros_v2_bringup'), 'config'),
                '-configuration_basename',
                'cartographer_2d_localization.lua',
                '-load_state_filename', state_filename_file],
            remappings = [('imu', '/imu/data_raw')],
            output = 'screen'
        ),
        Node(
            package = 'cartographer_ros',
            executable = 'cartographer_occupancy_grid_node',
            parameters = [
                {'use_sim_time': False},
                {'resolution': 0.05}],
        )
    ]
)
```